

Course_Note

2025년 6월 18일 수요일 오후 5:36

❖ 다양한 유형의 파일 형식 이해

- Delimited text file : 구분자 형식 텍스트 파일
- CSV : Comma-separated values
- XLSX : 안전한 파일 형식
- XML : 확장 마크업 언어
 - 플랫폼과 프로그램 언어에 구애받지 않음
 - 인터넷 웹 위주, HTML과 유사하지만 태그 사전 정의하지 않음
- JSON : JavaScript Object Notation. 웹을 통해 데이터를 전송
 - 프로그램 언어에 구애받지 않음
 - 많이 상용화됨

❖ 클라우드 : 구독을 통한 컴퓨팅 자원을 제공. servers, storage, applications, services, and data centers 등

❖ 클라우드 서비스 모델

- IaaS : Infrastructure as a service
- PaaS : Platform as a service
- SaaS : Software as a service

❖ 빅데이터 처리 도구

- Hadoop : 데이터를 저장할 수 있는 안정적이고 확장가능하게 함
 - HDFS : 분산파일시스템
- Hive : 데이터 웨어하우스 소프트웨어로 데이터 파일을 읽고, 쓰고, 관리.
 - 느리고 읽기 위주. SQL이나 데이터 분석 위주
- Spark : 범용 데이터 처리 엔진
 - 메모리 내 처리
 - 스트리밍 데이터 처리 및 실시간 분석

❖ 데이터마이닝 과정

❖ STEM classes : Science, Technology, Engineering, and Mathematics (STEM)

❖ Generative AI

- 두 개의 근본이 되는 요소
 - GANs : Generative adversarial networks
 - VAEs : Variational auto-encoders
- 데이터사이언티스트는 데이터셋을 보강하기 위해 합성데이터를 생성할 수 있음
- IBM Cognos Analytics : AI 기반 자동화

❖ 신경망 및 딥러닝

- 이해하기 위해서는 선형 대수학을 배워야 함
- 패키지를 쓰되 기법을 이해해야 함

❖ 데이터 유형

- 정형 데이터 : 데이터베이스에 저장 가능
- 반정형 데이터 : JSON, XML 등 계층적 데이터
- 비정형 데이터 : NoSQL 등 행과 열로 저장되지 않음
- Flat file : CSV 등, spreadsheet file은 flat file의 다른 종류
 - XML : 계층 정보 포함 가능
 - XML 이후 JSON이 대안으로 활용됨. 미리 정의된 스키마가 없어 여러 데이터 구조 간에 쉽게 전송 가능
- API : Application Program Interfaces. 요청을 받아 여러 file 형태로 데이터를 반환함
 - 조직 내부 및 외부의 데이터베이스 소스에서 데이터를 가져오는 데 사용
 - JSON의 경우 RESTful API로 데이터를 전송함
- Web Scraping : 비정규화된 구조에서 데이터 추출
 - 대표적 툴 : BeautifulSoup, Scrapy, Pandas, Selenium
- Data Streams and feeds : 여러 매체, 웹사이트 등에서 데이터 흐름을 축적하는 것
 - 일반적으로 타임스탬프와 지리적 태그 지정됨
 - 대표적 툴 : Kafka, Apache Spark, Apache Storm

❖ 메타 데이터는 데이터 카탈로그에 저장됨

❖ Data Management

- MySQL
 - RDBMS로 웹 애플리케이션, 데이터 웨어하우징, 전자상거래 등에 활용
- PostgreSQL
 - JSON 지원하고 공간 데이터에 유용
- mongoDB
 - 문서 지향 NoSQL으로 JSON으로 데이터 저장하고 확장성과 데이터 분산 기능으로 대량의 비구조적 데이터를 처리하는 현대 웹 애플리케이션에 적합함
- apache couchDB
 - 문서 지향 NoSQL으로 JSON 사용함
- apache cassandra
 - 문서 지향 NoSQL으로 많은 일반 서버에 거쳐 대량의 구조화된 및 비구조화된 데이터 처리 가능
- hadoop HDFS
 - 파일을 블록으로 나누어 이 블록을 여러 데이터 노드에 분산함. 높은 데이터 처리량
- Elasticsearch : 색인 생성을 포함하여 텍스트 데이터를 저장함
 - 주로 분산 RESTful 검색 엔진 및 분석 도구
 - 전체 텍스트 검색 및 실시간 데이터 분석에 용이

❖ ETL (Data integration and transformation)

- 데이터 정제 포함
- apache Airflow
 - Airbnb에 의해 생성되었고 프로그램적으로 워크플로를 작성하고 예약 및 모니터링 가능
- Kubeflow : Kubernetes 위에서 데이터 사이언스 파이프라인을 실행함
- apache kafka

- 애플리케이션이 실시간으로 레코드 스트림을 게시하고 처리할 수 있도록 하는 분산 스트리밍 플랫폼
- LinkedIn에 의해 개발되었고 확장 가능하며 내결함성이 있음.
- Apache NiFi : 시스템 간 데이터 흐름을 자동화. 데이터 라우팅, 변환 등
 - 데이터 라우팅? 네트워크에서 데이터 패킷이 출발지에서 목적지까지 전달될 때 최적의 경로를 선택하고 결정하는 과정
- apache SparkSQL : 수 천개의 노드로 구성된 클러스터를 계산하도록 확장가능. 다양한 데이터 소스 지원

❖ Data Visualization

- 간단한 툴들 : Tableau, PowerBI
- PixieDust : 파이썬 및 주피터 노트북에서 사용
- Hue(Hadoop 시각화) : 대규모 데이터 세트 분석 및 시각화. SQL 쿼리로 시각화 가능
- Kibana : 웹 기반 인터페이스, Elasticsearch와 함께 일반적으로 사용됨
- Apache Superset : 현대적인 기업용 비즈니스 인텔리전스 웹 애플리케이션

❖ Model deployment : 모델 배포. 다른 개발자가 사용할 수 있게 API로 전환

- Apache PredictionIO
- Kubernetes : 컨테이너화된 애플리케이션을 자동으로 시작, 확장, 관리
 - 여러 호스트에 걸쳐 컨테이너 관리 및 오케스트레이션
- Apache Seldon : Kubernetes에서 배포하고 관리. 워크플로 자동화 및 성능 실시간 모니터링
- MLeap
- TensorFlow Serving : 모바일 및 임베디드 장치에서 머신러닝 모델을 실행

❖ 모델 모니터링 : 배포된 모델 추적함. Fiddler 등

- ModelDB : 실험 추적 및 재현, 모델 버전 관리 및 팀원과 협업
- Prometheus : 다양한 출처에서 실시간 메트릭 수집 및 저장
- IBM AI Fairness 360 : 머신러닝 모델에서 편향을 감지하고 완화하는 툴킷
- IBM AI Explainability 360 : 모델의 행동과 결정을 설명하는 툴킷
- IBM Research Trusted AI

❖ 코드 개발 및 실행

- Jupyter IDE
- Rstudio
- Microsoft Visual Studio
- Pycharm
- Spyder
- Anaconda Navigator

❖ 코드 자산 관리 도구

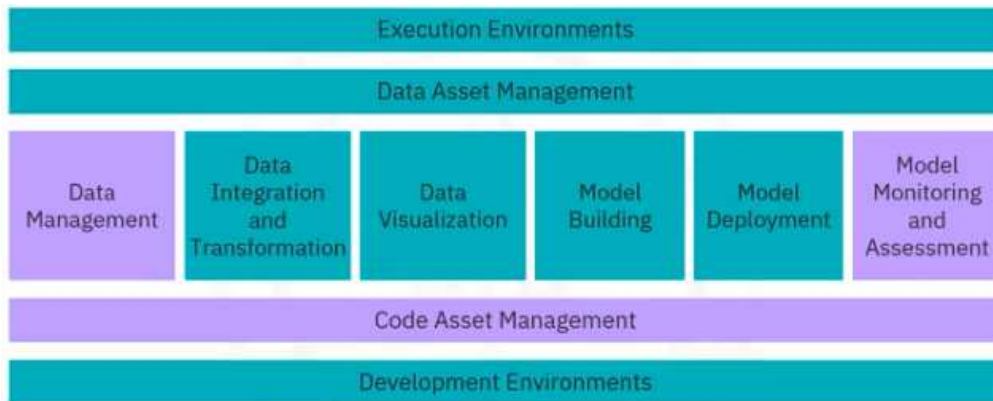
- Git
- GitLab
- GitHub

❖ 모델 평가 : F1 score나 SSE 등

❖ Code Asset Management : 인벤토리를 관리하는 통합 뷰. 버전을 통해 변경을 추적함

- 툴 예시 : Git, GitHub(버전 관리, 버전 제어 등), GitLab

- ❖ DAM (Data Asset Management) : 데이터를 정리하고 관리하는 플랫폼
 - 톨 예시 : Apache Atlas, Egeria and Informatica(IBM)
- ❖ IDEs (Integrated Development Environments) : 메타데이터 관리 및 운영

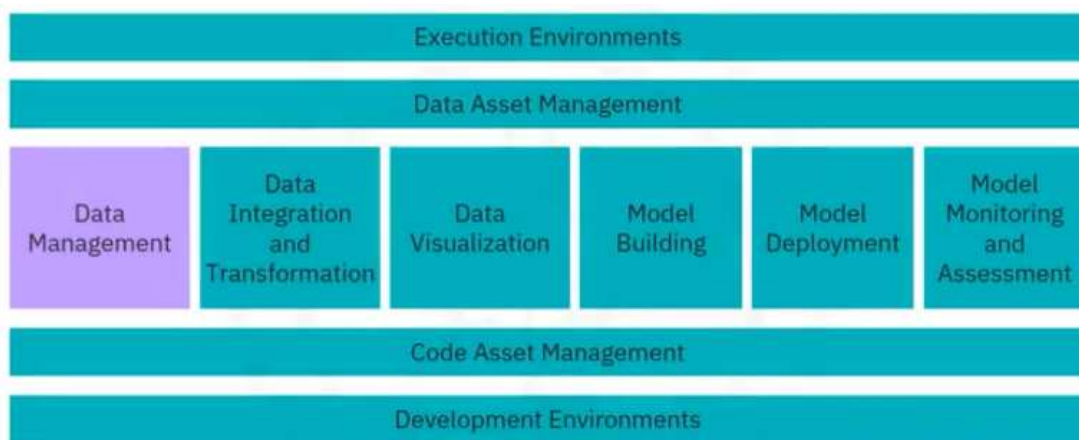


- ❖ 모델 빌딩 tool : SPSS Modeler, SAS enterprise miner

-
- ❖ Jupyter : 커널을 통해 다양한 프로그래밍 언어 제공. 캡슐화
 - ❖ Jupyter Lab : 차이점은 파일과 터미널, 데이터셋 등 여러 파일을 캔버스에 정렬 가능함
 - ❖ Apache Zeppelin : 통합 플로팅 기능. 코딩이 필요 없음
 - ❖ R studio : R 언어 기반, 프로그래밍, 실행, 디버깅, 원격 데이터 액세스, 데이터 탐색과 시각화를 하나의 도구로 통합
 - ❖ Spyder : R studio를 모방하여 python으로 변환하고자 한 것. 그럼에도 jupyter 더 많이 사용함
 - ❖ Microsoft Visual Studio : 여러 프로그램 언어를 지원하는 IDE
 - ❖ Apache Spark : 데이터 저장소 및 메모리 확보를 위한 클러스터 실행 환경. 선형 확장성이 있음
 - ❖ Apache Flink : 실시간 데이터 스트림을 처리하는 데 중점을 둔 스트림 처리 이미지
 - ❖ Fully Integrated Visual Tools : 프로그래밍 지식 필요 없음. ETL, 시각화, 모델 구축의 모든 작업 포함
 - 예시 : KNIME, Orange

-
- ❖ Fully Integrated Visual Tools (Cloud service)
 - IBM Watson Studio
 - Azure Machine Learning : 클라우드 기반 데이터 사이언스의 life cycle 지원
 - H2o.ai
 - ❖ Data management (Cloud service)
 - SaaS : Software as a service
 - 클라우드 공급자가 클라우드에서 도구를 대신 운영함
 - Amazon DynamoDB, Cloudant(couchDB 기반), DB2(IBM)
 - 업데이트, 백업, 복원 및 확장 작업을 클라우드 공급자가 수행함
 - ❖ ETL or ELT (Cloud service)
 - Informatica
 - IBM Data Refinery
 - IBM Watson Studio의 일부
 - Raw data를 스프레드시트 형태로 변환 가능
 - ❖ Data visualization (Cloud service)
 - Datameer

- IBM Cognos Analytics 이나 IBM Data Refinery도 시각화 기능 제공
- ❖ Model building (Cloud service)
 - IBM Watson Machine Learning : 다양한 오픈소스 라이브러리 활용 모델 학습
 - Google Cloud
- ❖ Model deployment (Cloud service)
 - 모델 빌딩 과정과 긴밀히 통합됨
 - SPSS Modeler는 예측 모델 마크업 언어(PML)로 내보내는 것을 지원함
 - IBM Watson Machine Learning도 REST 인터페이스 사용하여 모델을 사용할 수 있게 함
 - REST 인터페이스 : 웹에서 데이터를 주고받는 규칙이자 설계 방식
- ❖ Model monitoring and assessment (Cloud service)
 - Amazon Sagemaker Model Monitor
 - Watson OpenScale
- ❖ 아래의 초록색 부분은 Watson studio와 opensale을 통해 전부 커버 가능함. 전체 개발 주기

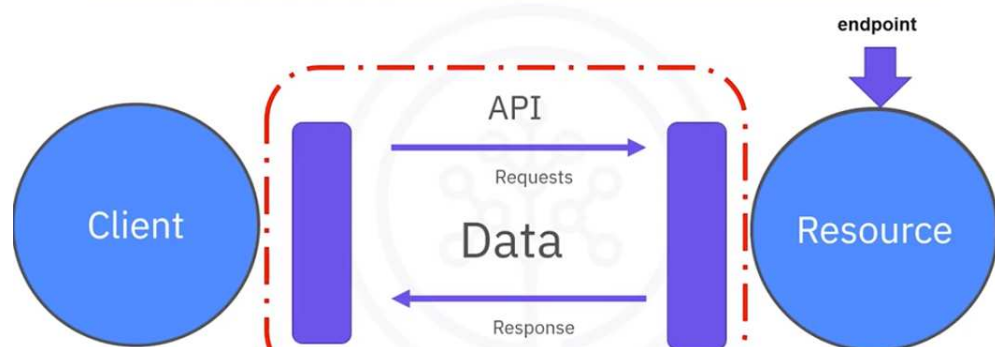


- ❖ Python
 - 라이브러리
 - 자주 사용 : Pandas, Numpy, Scipy, Matplotlib
 - AI 관련 : TensorFlow, Pytorch, Keras, Scikit-learn
 - 컴퓨팅 관련
 - Pandas : 데이터 구조 및 툴, 데이터프레임 기반으로 작업하며 인덱싱 기능
 - Numpy : 행렬, 매트릭스, 수학적 함수 적용 가능
 - 시각화 관련
 - Matplotlib : plot, graph
 - Seaborn : matplotlib 기반으로 히트맵, 시계열, violin plot 등
 - 머신러닝 관련
 - Scikit-learn : 회귀, 분류, 클러스터링

- Keras : 딥러닝
 - 딥러닝 관련
 - Tensorflow : 생산과 배포용
 - Pytorch : 간단한 실험용으로 용이함
 - NLTK로 NLP(자연어 처리) 사용 가능
- ❖ SQL
 - 비절차적 언어
 - 언어를 사용가능한 SQL 데이터베이스들



- ❖ API : Application Programming Interfaces
 - 두 애플리케이션 간의 통신을 가능하게 함
 - 예를 들어 pandas API로 다른 소프트웨어 구성요소와 통신하여 데이터 처리
 - REST APIs : Representational State Transfer APIs
 - 인터넷으로 소통하며 스토리지, 데이터, 알고리즘 등의 소스를 활용할 수 있게 함



- 데이터는 HTTP 방법을 사용하여 인터넷을 통해 전송됨
 - 요청은 JSON 파일이 포함된 HTTP 메시지를 사용하여 전송됨
 - 파일에 웹서비스에서 수행할 작업에 대한 지침이 들어 있음
 - 마찬가지로 웹 서비스(Resource)가 HTTP 메시지를 통해 응답을 반환
 - API 응용분야 : 다른 시스템의 데이터와 기능에 접근할 수 있는 방법을 제공
 - 소셜미디어 플랫폼
 - 전자상거래 웹사이트
 - 날씨 애플리케이션
 - 지도 및 내비게이션 애플리케이션
 - 결제 게이트웨이
 - 메시징 애플리케이션
- ❖ 데이터셋
 - 형태

- Tabular data
- Hierarchical data
- Raw file : 이미지
- UN 오픈데이터 : <https://data.un.org/>
- Kaggle 오픈데이터 : <https://www.kaggle.com/data> sets
- Google data set search : <https://data.setsearch.research.google.com/sets>
- ❖ 머신러닝
 - 종류
 - 지도학습
 - 회귀 : 수를 예측
 - 분류 : 데이터를 카테고리화, 클래스화
 - 비지도학습
 - 클러스터링 : 유사한 그룹끼리 묶기
 - 이상치 식별
 - 강화학습
 - 인간이 학습하는 방식과 유사
 - 보상을 받기 위해 취해야 할 최고의 조치
 - 딥러닝
 - 활용 : 자연어 처리, 이미지, 오디오, 영상, 시계열 예측
 - 모델
 - ◆ 처음부터 딥러닝 모델 구축하거나 공개 모델 리포지토리에서 사전학습된 모델 사용
 - ◆ 프레임워크 Tensorflow, Pytorch, Keras 등 활용하여 구현

- ❖ MAX (Model Asset Exchange) : IBM 개발자 플랫폼의 모델 자산 거래소, 딥러닝 관련
 - 사전학습(ready-to-use)되었거나 custom 모델을 사용하여 비즈니스 문제를 해결
 - 허가형 오픈 라이선스로 제공됨
 - MAX는 시간을 단축시키는 효과가 있음
 - Typical model-serving microservice 구성요소들
 - 사전학습된 딥러닝 모델
 - 사전 프로세스 input code
 - 사후 프로세스 putput code
 - 애플리케이션 연결을 위한 공개 API
 - GITHUB : <https://github.com/CODAIT/max-central-repo>
 - MAX model-serving microservice
 - 오픈 소스 docker 이미지로 구축, 배포됨
 - Docker : 애플리케이션을 쉽게 구축하고 배포하는 컨테이너 플랫폼
 - 도커 이미지는 Github에 게시되고 사용자들은 다운하여 특정 환경에서 사용하도록 customize 할 수 있음
 - Kubernetes 시스템을 통해 배포, 확장, 관리를 자동화할 수 있음
 - 따라서 Docker 또는 Kubernetes를 사용하여 마이크로서비스로 실행할 수 있음

- ❖ CodePen 플랫폼에 호스팅된 객체 탐지기 모델 실습
 - CodePen : 소셜 개발 환경으로 브라우저에서 코드 빌드에 따른 결과 확인 가능
-

❖ Jupyter Notebooks

- Julia, Python, R을 지원함
- 코드, 방정식, 시각화, 설명 텍스트 등이 포함된 문서를 만드록 공유할 수 있는 브라우저 기반 애플리케이션
- 과학자의 실험 공책
- PDF나 HTML 포맷으로 공유 가능함

❖ Jupyter Lab

- 다양한 주피터 노트북 파일이나, 코드, 데이터 파일에 접근가능하도록 함
- 텍스트 편집기, 터미널 등 확장하여 사용할 수 있도록 함. 여러 파일 형식과 호환됨
- 클라우드 기반 서비스인 IBM, Google Colab 등이랑 사용 가능, 설치 필요 없음

❖ 설치 방법

- 명령창에 `pip install jupyterlab`
- 아나콘다 닷컴의 아나콘다 플랫폼을 통해 로컬로 다운
 - 아나콘다는 jupyter를 포함하는 인기 있는 배포판 중 하나

❖ Skills Network Labs (SN Labs) : 가상 실습 환경 <https://labs.cognitiveclass.ai/v2/tools/jupyterlite?ulid=ulid-759c2cbd6458cc7c606bf99317e6f6a5d13f7480>

❖ Kernal

- Python kernal은 파이썬을 실행할 수 있게 함
- SNLab에는 이미 몇 가지 언어가 사전설치되어 있음
- 로컬 환경에서는 명령줄 인터페이스(CLI)로 언어를 수동으로 설치해야 함

❖ Jupyter Architecture

- 2-process model : kernel과 client
- Kernel이 Notebook 안의 코드를 실행
- NB convert tool로 파일을 다른 형식으로 변환

❖ JupyterLab와 VSCode를 통해 JupyterNotebook 형식을 로컬 환경에서 만들고 수정함

❖ Anaconda

- Open-source distributor
- 1500+ 라이브러리들
- Anaconda navigator에서 jupyterlab 다운 가능

❖ Google Colab

❖ NB-Viewer : 주피터 파일 URL을 붙여넣어 파일 확인, <https://nbviewer.org/>

❖ R

- 추가 라이브러리 설치할 필요 없이 포함되어 있음
- Rstudio는 IDE 통합개발환경
- 여러 탭들이 있음
 - Code editor
 - Console

- Workspace history tab
 - Files, plots, packages, help
 - 사용가능한 라이브러리들
 - Dplyr : 데이터 조작
 - Stringr : 문자열 조작
 - Ggplot : 데이터 시각화
 - Caret : 머신러닝
 - 시각화 패키지
 - Ggplot : 히스토그램, 바 차트, 산점도
 - Plot에 계층을 추가하여 복잡한 요청을 처리함
 - Plotly : 웹기반 데이터 시각화. HTML 파일 표시
 - Lattice
 - Leaflet
 - 패키지 설치 방법 : `install.packages <package name>`
-

❖ Version Control : 문서의 변경 내용을 추적

- 실수 시에 이전 버전에 접근 가능
- 공동작업 수월

❖ Git : 분산 버전 제어 시스템

- 주로 개발 중에 소스 코드를 추적하는데 중점을 둠
- 프로그래머들 사이에 변경사항을 추적하고 비선형적인 워크플로우를 지원
- 애자일 개발 방법론에 중점을 둔 협업의 중심점 역할
- 별도의 브랜치에서 동시에 작업할 수 있음

❖ Git 명령어

- 새 repository 만들 때 : `git init`
- 변경사항을 작업 디렉토리에서 준비영역으로 이동 : `git add`
- 작업 디렉토리에서 변경내용의 준비된 스냅샷을 볼 수 있음 : `git status`
- 스테이지된 변경 내용 스냅샷을 가져와 프로젝트에 커밋 : `git commit`
- 작업 디렉토리의 파일에 대한 변경 내용을 취소 : `git reset`
- 프로젝트에 대한 이전 변경 내용 찾아봄 : `git log`
- Repository 내에 격리된 환경을 만들어 변경함 : `git branch`
- 기존 branch를 보고 변경함 : `git checkout`
- 모든 항목을 다시 한 번 통합 : `git merge`

❖ GitHub : Git 리포지토리에 가장 많이 사용되는 웹 호스팅 서비스

- SSH protocol : 한 컴퓨터에서 다른 컴퓨터로 원격 로그인을 보호하는 방법
- Repository : 버전 제어용으로 설정된 프로젝트 폴더
- Fork : Repository의 복사본
- Pull request : 변경이 완료되기 전에 다른 사용자가 변경사항을 검토하고 승인하도록 요청하는 방법

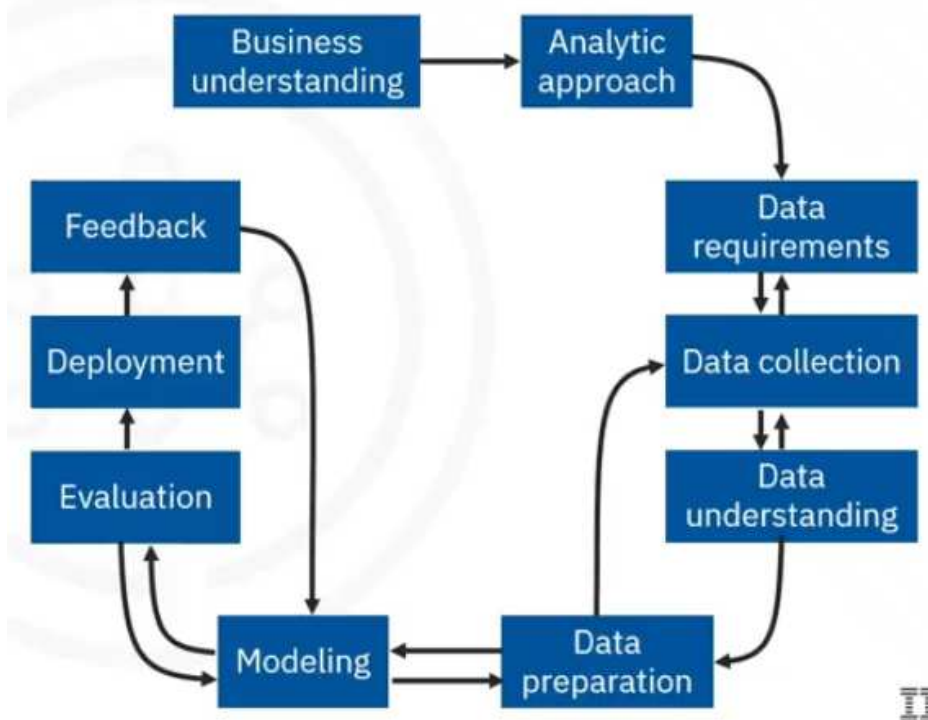
❖ Branch

- 변경할 수 있는 레포지토리의 스냅샷
- 마스터 브랜치의 복사본, 마스터 브랜치에 병합하기 전에 워크플로우 변경을 개발하고 테스트하는데 사용
- Master branch : 프로젝트의 정식 버전
- Child branch : 마스터 브랜치의 복사본. 빌드, 편집, 변경사항 테스트 수행
 - 만족하면 마스터 브랜치에 병합하여 모델을 배포할 준비
- PR (Pull Request) : 하위 브랜치의 내용을 병합하여 마스터 브랜치에 반영하는 법
 - 메인 브랜치의 변경 및 수정사항을 다른 팀원에게 알릴 수 있음 그리고 승인
 - "compare and pull request" 버튼을 통해 확인가능

❖ Watsonx.ai

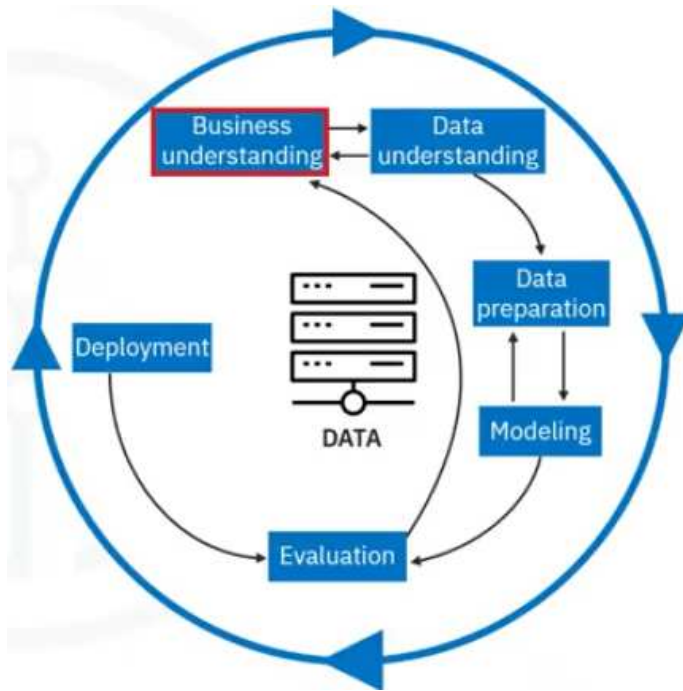
- 기능
 - Create projects
 - Upload files
 - Refine data
 - Create and share dashboards
 - Knowledge catalogs
 - Machine learning
 - Generative AI
- Cloud Pak for data : 데이터 액세스 및 통합 플랫폼
 - 다른 데이터 서비스와 함께 watsonx.ai, IBM knowledge Catalog, IBM Watson Machine Learning 등 사용 가능

❖ Data Science methodology questions



- ❖ Business understanding
- ❖ Analytic approach
- ❖ Data requirements
- ❖ Data collection

- ❖ Data understanding
- ❖ Data preparation : cleansing time, transforming data
 - Feature engineering
 - Working with text analysis
- ❖ Modeling : Using predictive or descriptive, using training/test sets
 - Understand the question
- ❖ Evaluation
 - Diagnostic measures
 - Statistical significance
- ❖ Deployment
- ❖ feedback
- ❖ Descriptive Analysis : current status
 - Data aggregation : combining data
 - Data mining : extracting information
 - Data visualization
 - Ex : creating dashboards
- ❖ Diagnostic Analytics : why did it happen
 - Drill-down : explore detailed data
 - Data discovery : identify patterns and relationships
 - Correlation analysis
 - Ex : root causes of sale decline
- ❖ Predictive Analysis
 - Regression analysis
 - Time series forecasting
 - Machine learning models
- ❖ Prescriptive questions : what should we do
 - Optimization models : finding best solution from the alternatives
 - Simulation : modeling scenarios
 - Decision analysis : evaluating and comparing
- ❖ Classification questions : which category this belong to
 - Logistic regression
 - Decision trees
 - Support vector machines
 - Neural networks
- ❖ **CRISP-DM 모델**
 - Cross-Industry Standard Process for Data Mining



- 순환적 6단계
 - Business understanding
 - Data understanding : data requirements, data collection, data understanding
 - Data preparation
 - Modeling : data mining
 - Evaluation
 - Deployment : 새 데이터에 모델을 적용
- 다시 결과에 대해 이해관계자들과 회의

❖ Changing expression type

- Float(1) = 1.0
- Int(1.1) = 1
- Int(False) = 0

❖ String 분할

- Ex : Michael Jackson
 - Name[::2] : "McalJcsn"
 - Name[0:5:2] : "Mca"
- Escape Sequences
 - \ : escape 시퀀스를 진행함
 - Print("Michael Jackson \n is the best") : 새로운 줄 출력
 - Print("Michael Jackson \t is the best") : 탭

❖ B = A.upper()

❖ B = A.replace('Michael', 'Janet')

❖ Name.find('el') : 5

❖ Format String

- Data :
 - name = "John"
 - age = 50
- f-strings
 - print(f"My name is {name} and I am {age} years old.")
- str.format()

- `print("My name is {} and I am {} years old.".format(name, age))`
- % 연산자
 - `print("My name is %s and I am %d years old." % (name, age))`
- ❖ `name.split()`
- ❖ `re.search(pattern, s1)`
- ❖ `matches = re.findall(pattern, text)`
- ❖ **Tuple**
 - 변경할 수 없음. 새 tuple을 만들어야 함
 - `RatingsSorted = sorted(Ratings)`
 - Nesting : 다양한 데이터 타입 혼합 가능
- ❖ **Lists**
 - 차이점은 변경 가능하다는 것
 - 리스트안에 튜플, 리스트 등 모두 넣을 수 있음
 - 리스트+리스트로 추가 가능
 - `L.extend(["pop", 10])` : 기존 리스트에 2개의 요소 추가
 - `L.append(["pop", 10])` : 기존 리스트에 1개의 원소(리스트) 추가
 - `Del(A[0])`
 - `"hard rock".split()` : 문자열을 요소로 이루어진 리스트로 변환
 - `"A,B,C,D".split(",")`
 - Aliasing : 동일한 객체를 참조하는 여러 변수(이름), B=A
 - 따라서 한 변수에서 객체를 변경하면 다른 변수에서도 변경된 것으로 표시됨
 - Clone : `B = A[:]`
- ❖ **Dictionaries**
 - Keys : 변경 불가능하고 고유
 - Values : 값은 변경 가능하거나 불가능, 중복될 수 있음
 - `Del(DICT["Thriller"])`
 - "The Bodyguard" in DICT : 요소(키)가 사전에 있는지 확인
 - `DICT.keys()` : 모든 키 확인
 - `DICT.values()`
- ❖ **Sets**
 - 집합
 - 순서가 지정되지 않음. 위치가 존재하지 않고 고유한 값으로 구성
 - 중복 항목은 반영되지 않음
 - `A.add("NSY")`
 - `A.remove("NSY")`
 - "AC" in A
 - 교집합 : `set3 = set1 & set2`
 - `Set1.intersection(set2)`
 - 합집합 : `set1.union(set2)`
 - 차집합 : `set1.difference(set2)`
 - 하위집합인지 확인 : `set3.issubset(set1)`
- ❖ **Functions**
 - Sorted vs sort

- Sorted_rating = sorted(ratings)
- Ratings.sort()

```
def ArtistNames (*names):
    for name in names:
        print(name)
```

names=("Michael Jackson","AC/DC","Pink Floyd")

ArtistNames("Michael Jackson","AC/DC","Pink Floyd")

- Pass
 - 임시 자리 표시자. 함수나 조건 블록을 정의할 때 코드가 아무 작업도 하지 않더라도 문법적으로 올바르게 유지되도록 함
 - 코드를 실행하지 않고 단순히 다음 문장으로 넘어감
- ❖ 예외 처리
 - Try and except 블록 : 프로그램 중단 방지
- ❖ Python은 객체지향 프로그래밍(OOP) 언어
 - 객체와 클래스를 중심으로 한 패러다임을 사용함
- ❖ Class 정의하기
 - 클래스 이름과 object를 지정
 - 클래스 속성 : 모든 클래스 객체 간에 공유되는 변수 정의
 - Class_attribute = value
 - 생성자(__init__) 활용하여 각 인스턴스를 초기화
 - 생성자는 새 클래스를 만들고 있다고 알려주는 특수 함수
 - Self parameter: 새로 만든 클래스의 인스턴스를 나타냄
 - Self를 객체의 모든 데이터 속성을 포함하는 상자라고 생각
 - Methods 지정
 - Self 매개변수는 인스턴스 method에서 필수로 요구되면, 이를 통해 인스턴스 속성에 접근하고 다른 method를 호출할 수 있음
- ❖ 객체 인스턴스화 : my_car = Car()

❖ Open으로 파일 열기

- 1번째 매개변수 : 파일 경로
- 2번째 매개변수 : 모드
 - 'r': 읽기용 [주로 사용]
 - 'w': 쓰기용
 - 새로운 파일 생성 또는 덮어쓰기 가능
 - 기존 내용을 바로 덮어 쓰므로 내용 추가시 append 사용
 - 'a': append용
 - 새로운 파일 생성하지 않고 기존 파일에 추가

- 'r+' : 읽기, 쓰기, truncate 불가
 - .truncate() 추가하여 기존 코드의 내용들 삭제
- 'w+' : 쓰기, 읽기, truncate 가능
- 'a+' : 더하기, 읽기
- With 문 사용하여 파일을 열면 자동으로 파일 닫힘
 - With open("Example.txt", "r") as file1:
 - 들여쓰기를 통해 read()하고 print
- ❖ 포인터 특정 위치 설정
 - .tell() : 현재 위치 확인, 출력(print)
 - .format(file.tell())
 - .seek(offset, from) : 시작 위치로
 - .seek(0,0) # 0 byte만큼 시작에서 움직임
- ❖ Readlines() : 모든 문장을 리스트의 요소로 출력 가능
- ❖ File_stuff=file1.readline() # 한 문장 읽음
 - Print(file_stuff)
- ❖ Readline()의 매개변수값에 따라 출력할 글자 수 지정 가능
 - Readline(4) : 파일의 처음 4글자 출력
- ❖ 문서 Copy할 때
 - With open read
 - With open w
 - For ans 형식 활용가능

❖ Pandas 라이브러리

- Pandas의 빌트인 함수들
 - Df = pandas.read_csv('파일 경로')
 - Df = pandas.read_excel('파일 경로')
 - Df = pd.DataFrame(DIC) : 딕셔너리를 데이터프레임으로 변환
 - Key는 데이터프레임의 헤더(0번째 행)으로 구성됨
 - 기존 데이터프레임 기반 축소된 새 데이터프레임 생성
 - X = df[['Length']]
 - X = df[['Artist', 'Length', 'Genre']]
 - 고유 요소에 액세스하는 법 : iloc 방법
 - Df.iloc[0,0] : 'Michael Jackson'
 - Df_new.index = ['a', 'b', 'c'] : 기존 행의 인덱스들 새롭게 정의
 - 슬라이싱하여 새로운 df 할당
 - Y = df.loc['Jane', 'Department']
 - ◆ Df.loc[0:2, 'ID':'Department']
 - Z = df.iloc[0:2, 0:3]
 - 단, 0번째에서 2번째 끝까지 모든 값 포함됨. 따라서 0, 1, 2번째로 구성

- 특정 속성에서 고유한 값들 찾기 : `df['Released'].unique()`
- `Df 1 = df[df['Released']>1979]` : 새로운 범위의 df 정의
- Df를 csv로 저장하기 : `df1.to_csv('new_songs.csv')`

❖ 1D Numpy 라이브러리

- `A = np.array([0,1,2,3,4])`
- Attribute 기본 배열 속성
 - Size : 배열의 요소 수
 - Ndim : 배열(array)의 차원 수, 배열의 랭크 수
 - Shape : 각 차원의 배열 크기
- Array 더하기 : `z = np.add(u, v)`
- Array 빼기 : `c = np.subtract(a, b)`
- Array 곱하기 : `z = np.mulitply(x, y)`
- Array 나누기 : `z = np.divide(x, y)`
- Transpose : `C.T`
- For n,m in zip(u,v):
 - `z.append(n+m)`
- Dot product : `uTv`
 - `Result = np.dot(u,v)`
- Universal Functions
 - `a.mean()`
 - `b.max()`
 - `Std() / min()`
 - `Y = np.sin(x)` # 사인함수 적용
 - 균일한 간격의 숫자 반환 : `np.linspace(-2, 2, num=5) : -2, -1, 0, 1, 2`
- Array slicing : `A[start:end:step]`
- 2차원 행렬 정의
 - `A = [[11,12,13], [21, 22, 23], [31, 32, 33]]`
 - `A.ndim : 2`
 - `A.shape : (3,3)`
 - `A.size : 9`
 - 대괄호 1개 vs 대괄호 2개 차이
 - `df['GDP (Million USD)']` : 하나의 열(column)만 선택할 때 사용
 - `df[['GDP (Million USD)']]` : 열이 1개인 데이터프레임 유지

❖ API

- Pandas API : 활용하면 다른 소프트웨어 구성 요소와의 통신으로 데이터를 처리할 수 있음
- REST API
 - 인터넷을 통한 통신이 가능
 - 스토리지와 같은 리소스 활용
 - Client(사용자 또는 사용자 코드)와 web service(resource)의 소통
- HTTP message : 인터넷을 통해 데이터를 전송하는 방법
 - REST API에 수행할 작업을 알려주기 위해 요청을 보냄, also 반환

- 주로 JSON 파일 포함 : 서비스에서 수행해야 할 작업에 대한 지침

❖ **URL** : Uniform Resource Locator, 웹에서 리소스를 찾는 방법

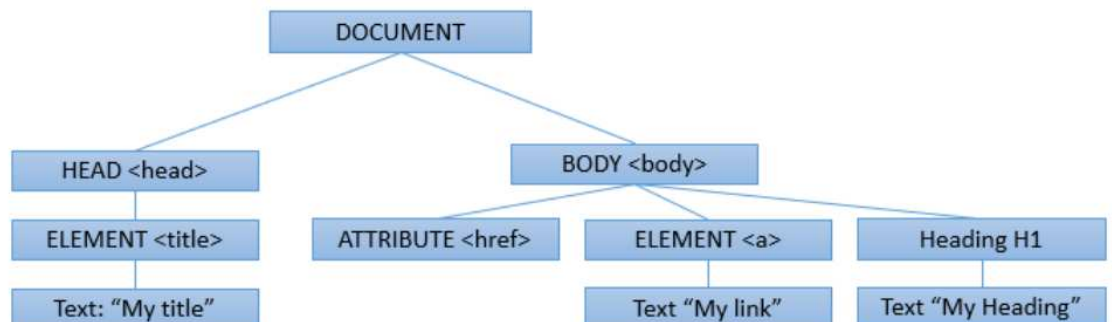
- 3가지 구성 요소
 - Scheme : 프로토콜, 예 - http://
 - Internet address or BaseURL : 경로를 찾기 위한 주소
 - Route : 웹 서버의 위치
- 요청 메시지 (Request Message)
 - Request Start line
 - Get 메소드, 파일을 요청
 - Request Header : 추가 정보 전달, 페이지에 대한 메타 정보 포함
 - Request Body
- 응답 메시지 (Response Message)
 - Response Start line : 버전 번호, 성공 메시지(200, OK)

Status Code

1XX	Informational
100	Everything So Far Is OK
2xx	Success
200	OK
3XX	Redirection
300	Multiple Choices

4XX	Client Error
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error
501	No Implemented

- Response Header : 추가 정보
- Response Body : 요청된 파일 내용



❖ HTTP methods

- GET : 서버에서 데이터 가져옴
- POST : 서버로 데이터 전송
- PUT : 서버의 기존 데이터를 업데이트
- DELETE : 서버에서 데이터를 삭제

❖ Request 모듈 (python)

- Requests 라이브러리 : HTTP/1.1 요청을 쉽게 전송할 수 있는 라이브러리
- `r=requests.get(url)` : 해당 URL로 GET 요청을 보냄
 - `r=requests.get(url_get, params=payload)`
 - `url_get='http://httpbin.org/get'`
 - 'r'은 응답 개체

- r.status_code 조회
- r.request.headers
- r.request.body
- header = r.headers
 - Header['date']
 - Header['Content-Type']
- r.text()
- r.json()
- POST 요청
 - URL이 아닌 요청 본문으로 데이터를 전송함
 - r_post=requests.post(url_post,data=payload)
- ❖ HTML 문단 태그
 - <html>
 - <head>
 - <body>
 - <h3>
 - <p> : paragraph
- ❖ HTML 테이블 정의
 - <table> : table creation 태그
 - <td> : 헤더 태그
 - <tr> : table row 태그
 - <td> : table data 태그

```

<table>
  <tr>
    <td> ROW 1 COLUMN 1 </td>
    <td> ROW1 COLUMN 2 </td>
    <td> ROW1 COLUMN 3 </td>
  </tr>
  <tr>
    <td>ROW2 COLUMN 1</td>
    <td> ROW2 COLUMN 2 </td>
    <td> ROW2 COLUMN 3 </td>
  </tr>
</table>

```
- ❖ 웹스크래핑
 - 대표 라이브러리들 : BeautifulSoup, Scrapy, Selenium or pandas.read_html()
 - pandas.read_html()은 표 형식만 가능
 - 2개 모듈 필요 : requests, beautifulsoup
 - 웹페이지의 HTML 콘텐츠를 가져오기
 - BeautifulSoup
 - HTML 콘텐츠를 트리와 같은 구조로 표현하여 쉽게 탐색
 - from bs4 import BeautifulSoup
 - Html = "--"
 - Html = requests.get("http://..").text
 - Soup = BeautifulSoup(html, 'html5lib')
 - links = soup.find_all('a') : 웹페이지의 특정 부분 탐색, 추출
 - For link in links:

- Print(link.text)
- Pandas의 read_html 함수 : 테이블에서 자동으로 데이터 추출

Read/Save Other Data Formats

Data Formate	Read	Save
csv	<code>pd.read_csv()</code>	<code>df.to_csv()</code>
json	<code>pd.read_json()</code>	<code>df.to_json()</code>
excel	<code>pd.read_excel()</code>	<code>df.to_excel()</code>
hdf	<code>pd.read_hdf()</code>	<code>df.to_hdf()</code>
sql	<code>pd.read_sql()</code>	<code>df.to_sql()</code>